

TOPIC 21 — Component Lifecycle Hooks

Angular Learning Series

Category: Angular Core | Difficulty: Beginner-Intermediate | Relevance: Critical

Section 0 — Difficulty & Priority Rating

- **Difficulty:** Beginner-Intermediate — the individual hooks are simple; knowing which hook to use for which task, and how they interact with Signals, OnPush, and input() signals takes real practice
- **Angular Relevance:** Critical — every Angular component goes through a predictable lifecycle; knowing when each hook fires is what separates components that work correctly from ones that have subtle timing bugs — loading data too early, accessing the DOM before it exists, or leaking resources

Section 1 — Explanation

The Employee Onboarding Analogy

Imagine a new employee joining a company. Their working life follows a predictable sequence:

- **Hired** → constructor — the paperwork is signed, the employee exists, but has not started work yet. Do not ask them to do anything complex yet — their desk is not set up. Inject services here only.
- **Briefed on their role** → ngOnChanges — they receive their job description and responsibilities (the input() values). Happens before their first day and every time their role changes.
- **First day starts** → ngOnInit — they are at their desk, settled in, ready to do real work. Load data, set up subscriptions, start processes.
- **Doing their job** → change detection cycles — Angular checks their work regularly.
- **Given access to the office layout** → ngAfterViewInit — the building is fully constructed, they can now see and use all the rooms (child components, DOM elements).
- **Leaving the company** → ngOnDestroy — final cleanup, hand over work, close accounts.

constructor	→	hired – exists but not ready
ngOnChanges	→	briefed – inputs received
ngOnInit	→	first day – start real work
(change cycles)	→	doing the job
ngAfterViewInit	→	office access – DOM/children ready
ngOnDestroy	→	leaving – clean up everything

In Plain English

- Lifecycle hooks = methods Angular calls at specific moments in a component's life — from creation to destruction
- constructor = called when Angular creates the class instance — before inputs arrive — for inject() calls only
- ngOnChanges = called when input() values change — before ngOnInit on first load — and on every subsequent change
- ngOnInit = called once after the first ngOnChanges — the right place for initialisation work
- ngAfterViewInit = called after the component's template and all child components are fully rendered — for DOM access

- `ngOnDestroy` = called just before Angular destroys the component — the right place for all cleanup
- Order matters = the hooks fire in a guaranteed order — knowing that order prevents timing bugs

Technical Terms

- `OnChanges` = interface requiring `ngOnChanges(changes: SimpleChanges)` — fires when bound `input()` values change
- `OnInit` = interface requiring `ngOnInit()` — fires once after first `ngOnChanges`
- `DoCheck` = interface requiring `ngDoCheck()` — fires on every change detection cycle — use sparingly
- `AfterContentInit` = fires after ng-content projected content is initialised
- `AfterViewInit` = interface requiring `ngAfterViewInit()` — fires after the component's view and child views are initialised
- `OnDestroy` = interface requiring `ngOnDestroy()` — fires just before the component is destroyed
- `SimpleChanges` = an object mapping each changed `input()` property name to a `SimpleChange` object containing `previousValue`, `currentValue`, and `firstChange`
- Note on signal inputs = `input()` signal functions bypass `ngOnChanges` entirely — use `effect()` or `computed()` to react to signal input changes instead

Section 2 — Modern Angular Pattern (v17+)

Lifecycle hooks are where every previous Angular topic connects to timing:

- Topic 20/21 (Subscriptions & `takeUntilDestroyed`) — subscriptions are set up in `ngOnInit` and cleaned up automatically via `DestroyRef`. `takeUntilDestroyed` replaces manual `ngOnDestroy` cleanup.
- Topic 22 (async pipe) — the async pipe subscribes when the template is first evaluated (after `ngOnInit` runs) and unsubscribes via its own internal `ngOnDestroy`.
- Topic 23 (Signals) — `effect()` must be called in the constructor because it needs injection context. Signal updates in `ngOnInit` work fine. `ngAfterViewInit` is where you access DOM elements. Signal `input()` functions bypass `ngOnChanges` — use `effect()` to react to signal input changes.
- Topic 24 (`OnPush`) — `ngDoCheck` fires on every change detection cycle. `ngOnChanges` only fires when the input reference changes — exactly the same rule as `OnPush` (mutation is invisible).

Bridging note: Lifecycle hooks are Angular's event system for a component's own existence. `ngOnInit` is when you start work, `ngOnDestroy` is when you stop. Everything between — subscriptions, HTTP calls, DOM access, Signal effects — has a correct hook to live in. Using the wrong hook causes timing bugs that are subtle and hard to diagnose.

Section 3 — Code Example

Example 1 — Complete lifecycle — all hooks in order

```
import {
  Component, OnChanges, OnInit, DoCheck,
  AfterContentInit, AfterContentChecked,
  AfterViewInit, AfterViewChecked, OnDestroy,
  SimpleChanges, inject, input
} from '@angular/core';

@Component({
  selector: 'app-lifecycle-demo',
  standalone: true,
  template: '<p>{{ title() }}</p>'
})
export class LifecycleDemoComponent implements
  OnChanges, OnInit, DoCheck,
  AfterContentInit, AfterContentChecked,
  AfterViewInit, AfterViewChecked, OnDestroy {

  title = input('');
  userId = input<number>(0);

  // ① CONSTRUCTOR – inject() calls only – no inputs yet
  constructor() {
    console.log('① constructor');
    // DO: inject services via inject()
    // DON'T: HTTP calls, DOM access, read signal inputs
  }

  // ② NGONCHANGES – only fires for @Input() DECORATOR inputs, NOT signal inputs
  ngOnChanges(changes: SimpleChanges): void {
    if (changes['userId'] && !changes['userId'].firstChange) {
      console.log('userId changed:', changes['userId'].currentValue);
    }
  }

  // ③ NGONINIT – runs once – right place for real initialisation
  ngOnInit(): void {
    console.log('③ ngOnInit – signal input values available here');
    // DO: HTTP calls, set up subscriptions, signal effects
    // DON'T: access DOM, access viewChild – not rendered yet
  }

  // ④ NGDOCHECK – runs on EVERY change detection cycle – use sparingly
  ngDoCheck(): void { }

  // ⑤ NGAFTERCONTENTINIT – once after ng-content is projected
  ngAfterContentInit(): void { }

  // ⑦ NGAFTERVIEWINIT – once after template + children rendered
  ngAfterViewInit(): void {
    console.log('⑦ ngAfterViewInit – DOM and child components ready');
    // viewChild IS available here
    // DO: access DOM elements, init third-party libraries
    // DON'T: update template-bound state – ExpressionChangedAfterItHasBeenChecked
  }
}
```

```

    }

    // ⑨ NGONDESTROY – just before component is removed
    ngOnDestroy(): void {
        // With takeUntilDestroyed/DestroyRef – often empty
    }
}

// LIFECYCLE ORDER (first load):
// ① constructor → ② ngOnChanges → ③ ngOnInit → ④ ngDoCheck
// → ⑤ ngAfterContentInit → ⑥ ngAfterContentChecked
// → ⑦ ngAfterViewInit → ⑧ ngAfterViewChecked
// ON DESTRUCTION: ⑨ ngOnDestroy

```

Example 2 — @Input() + ngOnChanges vs input() signal + effect()

```

// ---- APPROACH A: @Input() decorator + ngOnChanges ----
@Component({
    selector:      'app-user-detail-classic',
    changeDetection: ChangeDetectionStrategy.OnPush,
    standalone:    true,
    template: `
        @if (user()) { <h2>{{ user()?.name }}</h2> }
        @else { <p>Loading...</p> }
    `
})
export class UserDetailsClassicComponent implements OnChanges {

    @Input({ required: true }) userId!: number;

    user      = signal<User | null>(null);
    isLoading = signal(false);

    private readonly userService = inject(UserService);
    private readonly destroyRef  = inject(DestroyRef);

    ngOnChanges(changes: SimpleChanges): void {
        if (changes['userId']) {
            this.loadUser(this.userId);
        }
    }

    private loadUser(id: number): void {
        this.isLoading.set(true);
        this.userService.getUser(id).pipe(
            takeUntilDestroyed(this.destroyRef)
        ).subscribe({
            next: user => { this.user.set(user); this.isLoading.set(false); },
            error: () => { this.isLoading.set(false); }
        });
    }
}

// ---- APPROACH B: input() signal + toSignal + toObservable (modern) ----
@Component({

```

```

selector:      'app-user-detail-modern',
changeDetection: ChangeDetectionStrategy.OnPush,
standalone:    true,
template: `
  @if (userData()) { <h2>{{ userData()?.name }}</h2> }
  @else { <p>Loading...</p> }
`
})
export class UserDetailsModernComponent {

  userId = input.required<number>();

  private readonly userService = inject(UserService);

  readonly userData = toSignal(
    toObservable(this.userId).pipe(
      switchMap(id => this.userService.getUser(id).pipe(
        catchError(() => of(null))
      ))
    ),
    { initialValue: null as User | null }
  );

  // No ngOnChanges, no ngOnInit, no ngOnDestroy needed
  // toSignal manages subscription via DestroyRef internally
  // switchMap cancels previous HTTP call when userId changes
}

```

Example 3 — ngAfterViewInit — DOM and child component access

```

@Component({
  selector: 'app-chart',
  standalone: true,
  template: `
    <canvas #chartCanvas width="400" height="200"></canvas>
    <app-chart-legend #legend [data]="chartData" />
  `
})
export class ChartComponent implements AfterViewInit, OnDestroy {

  private readonly canvasRef = viewChild.required<ElementRef<HTMLCanvasElement>>('chartCanvas');
  private readonly legendRef = viewChild.required<ChartLegendComponent>('legend');

  chartData = [];
  private chart: ChartJS | null = null;
  private readonly destroyRef = inject(DestroyRef);

  ngAfterViewInit(): void {
    const ctx = this.canvasRef().nativeElement.getContext('2d');

    this.chart = new ChartJS(ctx, { type: 'bar', data: this.chartData });
    this.legendRef().highlight(0);

    this.destroyRef.onDestroy(() => {
      this.chart?.destroy();
      this.chart = null;
    });
  }
}

```

```

    });
  }

  ngOnDestroy(): void {
    // destroyRef.onDestroy() above already handles chart cleanup
  }
}

// IMPORTANT: Don't update template-bound state in ngAfterViewInit
@Component({ standalone: true, template: '<p>{{ title() }}</p>' })
export class SafeUpdateComponent implements AfterViewInit {

  title = signal('Initial');

  constructor() {
    // Fix 2 – afterNextRender() – Angular 17+ official solution
    afterNextRender(() => {
      this.title.set('Updated after render'); // always safe
    });
  }

  ngAfterViewInit(): void {
    // Fix 1 – defer to next microtask
    Promise.resolve().then(() => {
      this.title.set('Updated safely');
    });
  }
}

```

Angular Example — Production component using all correct hooks

```

@Component({
  selector: 'app-product-editor',
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    @if (isLoading()) { <div class="spinner">Loading...</div> }
    @if (product()) {
      <form [formGroup]="form" (ngSubmit)="onSubmit()">
        <input formControlName="name" placeholder="Product name">
        <input formControlName="price" type="number">
        <textarea formControlName="description"></textarea>
        <button type="submit" [disabled]="form.invalid || isSaving()">
          {{ isSaving() ? 'Saving...' : 'Save' }}
        </button>
      </form>
    }
    <canvas #previewCanvas width="200" height="200"></canvas>
  `
})
export class ProductEditorComponent
  implements OnChanges, OnInit, AfterViewInit, OnDestroy {

  @Input({ required: true }) productId!: number;

```

```

readonly product    = signal<Product | null>(null);
readonly isLoading  = signal(false);
readonly isSaving   = signal(false);

form!: FormGroup;

private readonly canvasRef      = viewChild<ElementRef<HTMLCanvasElement>>('previewCanvas');
private readonly destroyRef     = inject(DestroyRef);
private readonly productService = inject(ProductService);
private readonly fb             = inject(FormBuilder);
private readonly analytics      = inject(AnalyticsService);

constructor() {
  // ① effect() needs injection context – must be in constructor
  effect(() => {
    const p = this.product();
    if (p) this.analytics.track('product_viewed', { id: p.id });
  });
}

// ② fires on first load AND every productId change
ngOnChanges(changes: SimpleChanges): void {
  if (changes['productId']) this.loadProduct(this.productId);
}

// ③ one-time setup not dependent on @Input values
ngOnInit(): void {
  this.form = this.fb.group({
    name:      ['', Validators.required],
    price:     [0, [Validators.required, Validators.min(0)]],
    description: ['']
  });
}

// ④ DOM ready, viewChild available
ngAfterViewInit(): void {
  const canvas = this.canvasRef();
  if (canvas) this.initImagePreview(canvas.nativeElement);
}

ngOnDestroy(): void {
  this.form.reset();
  // takeUntilDestroyed handles subscription cleanup
}

private loadProduct(id: number): void {
  this.isLoading.set(true);
  this.productService.getProduct(id).pipe(
    takeUntilDestroyed(this.destroyRef)
  ).subscribe({
    next: p => { this.product.set(p); this.isLoading.set(false); },
    error: () => { this.isLoading.set(false); }
  });
}

```

```

onSubmit(): void {
  if (this.form.invalid) return;
  this.isSaving.set(true);
  this.productService.update(this.productId, this.form.value).pipe(
    takeUntilDestroyed(this.destroyRef)
  ).subscribe({ next: () => this.isSaving.set(false),
    error: () => this.isSaving.set(false) });
}

private initImagePreview(canvas: HTMLCanvasElement): void {
  canvas.getContext('2d')?.fillRect(0, 0, 200, 200);
}
}

```

Section 4 — Before & After Refactor

The Problem: The most common lifecycle mistake is putting everything in `ngOnInit` — including things that should react to input changes, things that need the DOM, and things that need cleanup. This creates a component that works on first load but breaks when inputs change, crashes when accessing `viewChild`, and leaks memory.

Before — everything crammed into `ngOnInit`

```

@Component({
  selector: 'app-user-profile',
  standalone: true,
  template: `
    <canvas #chart></canvas>
    @for (item of data; track item.id) { <div>{{ item.name }}</div> }
  `
})
export class UserProfileComponent implements OnInit, OnDestroy {

  @Input() userId!: number;
  @ViewChild('chart') chartRef!: ElementRef;

  data: Item[] = [];
  private chart: ChartJS | null = null;
  private sub: Subscription;
  private readonly dataService = inject(DataService);

  ngOnInit(): void {
    // ■ Accessing @ViewChild – not rendered yet – chartRef is undefined
    this.chart = new ChartJS(
      this.chartRef.nativeElement, // TypeError: cannot read nativeElement
      { type: 'bar', data: [] }
    );

    // ■ Won't re-run if userId input changes
    this.sub = this.dataService.getData(this.userId).subscribe(data => {
      this.data = data;
    });
  }

  ngOnDestroy(): void {

```

```

        this.sub?.unsubscribe(); // partial cleanup – chart not destroyed
    }
}

```

After — right hook for every task

```

@Component({
  selector:      'app-user-profile',
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone:    true,
  template: `
    <canvas #chart></canvas>
    @for (item of data(); track item.id) { <div>{{ item.name }}</div> }
  `
})
export class UserProfileComponent implements OnChanges, AfterViewInit, OnDestroy {

  @Input({ required: true }) userId!: number;

  private readonly canvasRef = viewChild<ElementRef<HTMLCanvasElement>>('chart');
  readonly data              = signal<Item[]>([]);
  readonly isLoading         = signal(false);

  private chart: ChartJS | null = null;
  private readonly destroyRef = inject(DestroyRef);
  private readonly dataService = inject(DataService);

  // ■ handles BOTH first load AND userId changes
  ngOnChanges(changes: SimpleChanges): void {
    if (changes['userId']) this.loadData(this.userId);
  }

  // ■ DOM is ready, viewChild available
  ngAfterViewInit(): void {
    const canvas = this.canvasRef();
    if (canvas) {
      this.chart = new ChartJS(
        canvas.nativeElement, // ■ rendered now
        { type: 'bar', data: { labels: [], datasets: [] } }
      );
    }
    this.destroyRef.onDestroy(() => {
      this.chart?.destroy(); // ■ third-party cleanup
      this.chart = null;
    });
  }

  ngOnDestroy(): void {
    // destroyRef.onDestroy handles chart cleanup
    // takeUntilDestroyed handles subscription cleanup
    // ngOnDestroy can be empty
  }

  private loadData(userId: number): void {
    this.isLoading.set(true);
    this.dataService.getData(userId).pipe(

```

```

        takeUntilDestroyed(this.destroyRef)
    ).subscribe({
        next: items => { this.data.set(items); this.isLoading.set(false); },
        error: () => { this.isLoading.set(false); }
    });
}
}

```

Why it matters: The before version has three bugs — `viewChild` accessed before the DOM exists (crash), `ngOnInit` not responding to input changes (stale data), and a leaked chart library (memory). The after version uses exactly the right hook for each task — no crashes, no stale data, no leaks.

Section 5 — Common Mistakes

Mistake 1 — "Put everything in `ngOnInit` — it runs after the component is ready"

`ngOnInit` runs after the first `ngOnChanges` — meaning inputs are available. But the DOM is NOT ready (`viewChild` is undefined), child components are NOT rendered, and it fires only ONCE — so it will not respond to input changes later.

```

ngOnInit(): void {
    // ■ Mistake 1: DOM access – not rendered yet
    this.canvasRef().nativeElement.focus(); // undefined

    // ■ Mistake 2: won't reload when input changes
    this.loadData(this.userId); // only runs once

    // ■ Mistake 3: third-party library needing the DOM
    this.map = new GoogleMap(this.mapContainer().nativeElement); // undefined
}

// ■ Each in the right hook:
ngOnChanges(changes: SimpleChanges): void {
    if (changes['userId']) { this.loadData(this.userId); }
}

ngAfterViewInit(): void {
    this.canvasRef().nativeElement.focus(); // ■ DOM ready
    this.map = new GoogleMap(this.mapContainer().nativeElement); // ■ DOM ready
}

```

Mistake 2 — "`ngOnChanges` only fires when the component is created"

`ngOnChanges` fires on first load AND every time any `@Input()` decorator value changes. If a parent changes a bound input — `userId` going from 1 to 2 — `ngOnChanges` fires again. Important: `ngOnChanges` only fires for `@Input()` decorator inputs — NOT for `input()` signal functions. For signal inputs, use `effect()` or `toObservable().pipe(switchMap(...))`.

```

// Parent changes userId input over time
@Component({ template: `<app-user [userId]="currentId" />` })
export class ParentComponent {
    currentId = 1;
    switchUser(): void { this.currentId = 2; } // triggers child's ngOnChanges
}

```

```
// Child – ngOnChanges fires BOTH times (with @Input() decorator)
@Component({ selector: 'app-user', standalone: true, template: '...' })
export class UserComponent implements OnChanges {
  @Input() userId!: number;

  private readonly userService = inject(UserService);
  private readonly destroyRef = inject(DestroyRef);

  ngOnChanges(changes: SimpleChanges): void {
    if (changes['userId']) {
      // Fires when userId = 1 (firstChange = true)
      // AND when userId = 2 (firstChange = false)
      this.loadUser(this.userId);
    }
  }

  private loadUser(id: number): void {
    this.userService.getUser(id).pipe(
      takeUntilDestroyed(this.destroyRef)
    ).subscribe(user => { /* update state */ });
  }
}
```

Mistake 3 — "Updating state in ngAfterViewInit is fine — the view is ready"

Angular runs change detection before ngAfterViewInit fires — the view has been evaluated and committed. Updating a template-bound property in ngAfterViewInit creates a mismatch and throws `ExpressionChangedAfterItHasBeenCheckedError` in dev mode.

```
// ■ Direct signal set in ngAfterViewInit
ngAfterViewInit(): void {
  this.title.set('Ready'); // ExpressionChangedAfterItHasBeenCheckedError in dev
}

// ■ Fix 1 – Promise.resolve() – defers to next microtask
ngAfterViewInit(): void {
  Promise.resolve().then(() => { this.title.set('Ready'); });
}

// ■ Fix 2 – afterNextRender() – Angular 17+ official solution
constructor() {
  afterNextRender(() => { this.title.set('Ready'); });
}
```

Section 6 — Do's and Don'ts

- DO: Use ngOnChanges for data loading driven by @Input() decorator inputs — handles first load and changes
- DON'T: Use ngOnInit alone for input-driven data loading — won't respond to input changes
- DO: Use ngOnInit for setup that doesn't depend on input values — form creation, non-input subscriptions
- DON'T: Access viewChild in ngOnInit — template not rendered yet — always undefined

- DO: Use `ngAfterViewInit` for DOM access, `ViewChild` usage, and third-party library initialisation
- DON'T: Update template-bound state directly in `ngAfterViewInit` — throws `ExpressionChangedAfterItHasBeenChecked`
- DO: Use `ngOnDestroy` or `destroyRef.onDestroy()` for third-party library cleanup
- DON'T: Leave `ngOnDestroy` empty for third-party libraries — Angular does not auto-destroy them
- DO: Implement lifecycle interfaces explicitly — implements `OnInit`, `OnDestroy`
- DON'T: Skip implementing interfaces — a typo in the method name silently never fires
- DO: Use `takeUntilDestroyed` / `DestroyRef.onDestroy()` — reduces what `ngOnDestroy` needs to do
- DON'T: Put all cleanup in `ngOnDestroy` — `DestroyRef` is more composable and testable

Section 7 — Interview Prep (Senior Level)

Question 1

The Question: "What is the difference between a component's constructor and `ngOnInit`, and what logic belongs in each?"

Strong answer: The constructor runs when Angular instantiates the class — before any inputs have been set and before the component is connected to the change detection tree. The only thing that belongs in the constructor is `inject()` calls for dependencies, and optionally `effect()` registration because `effect()` requires the injection context. `ngOnInit` runs once after the first `ngOnChanges`, meaning all `@Input()` decorator values and signal inputs are available. This is where data loading, form creation, and non-input-driven subscriptions belong. The constructor is about wiring the class. `ngOnInit` is about starting work.

Weak answer: "They both run at startup, but `ngOnInit` is safer because the component is ready."

Level: Mid

Question 2

The Question: "A user profile component loads data in `ngOnInit`. When navigating from `/users/1` to `/users/2`, the profile shows stale data. Why, and how do you fix it?"

Strong answer: Angular's router reuses the component instance when navigating between the same route with different params. `ngOnInit` fires only once on creation — it does not fire again on subsequent navigations to the same route with different params. The fix is to move data loading to `ngOnChanges` and check if (`changes['userId']`) — `ngOnChanges` fires on first load and every time the `@Input()` changes. For the modern signal-input pattern, use `toSignal(toObservable(this.userId).pipe(switchMap(id => this.userService.getUser(id))))` — this reacts to every `userId` change automatically, handles HTTP cancellation via `switchMap`, and manages the subscription lifecycle via `DestroyRef` internally.

Weak answer: "You need to unsubscribe and re-subscribe in `ngOnDestroy`."

Level: Senior

Section 8 — Quick Reference Card

3-line concept definition

Lifecycle hooks are methods Angular calls at guaranteed moments in a component's existence. Each hook has a specific purpose — using the wrong one causes timing bugs.

The order is fixed: constructor → ngOnChanges → ngOnInit → ngAfterViewInit → ngOnDestroy.

Which hook for what task

Task	Correct Hook	Common Mistake
-----	-----	-----
inject() services	constructor	Anywhere else
Load data based on @Input()	ngOnChanges	ngOnInit – won't re-run
One-time form/subscription setup	ngOnInit	constructor
Access viewChild / DOM element	ngAfterViewInit	ngOnInit – not rendered
Third-party library init	ngAfterViewInit	ngOnInit – no DOM yet
Subscription cleanup	takeUntilDestroyed	ngOnDestroy manual unsub
Third-party library cleanup	destroyRef.onDestroy	Forgetting entirely

When to use / when not to use

- Use ngOnChanges when: data loading depends on @Input() decorator inputs that can change over the component's life
- Use ngOnInit when: one-time setup (forms, non-input subscriptions) — runs after inputs are available
- Use ngAfterViewInit when: accessing the DOM, viewChild, or initialising third-party libraries that need a DOM element
- Avoid ngDoCheck: fires on every change detection run — extremely performance-sensitive — signals make it mostly unnecessary

The one rule that matters most

Wrong hook = silent timing bug. constructor = inject only. ngOnChanges = input-driven data. ngOnInit = one-time setup. ngAfterViewInit = DOM access. ngOnDestroy = explicit cleanup. Every other symptom — stale data, undefined viewChild, memory leaks — traces back to a hook mismatch.